# Laboratório Prolog · Aula 13

Gustavo Fumagalli Bandeira · RA 24011199-2 · Turma B
Gerado em 20/10/2026, 20:25:49 · tempo registrado no cronômetro: 0:00

## Preparação: Programar descrevendo, não instruindo

**Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?**

paradigma(L, funcional) dá L = scheme
paradigma(java, logico) dá false.

**Saída do ola.pl**

```
1. Saída:
?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(prolog,A).
   paradigma(prolog,logico)
?- paradigma(A,funcional).
   paradigma(scheme,funcional)
?- paradigma(java,logico).
   false.

2. Saída:
?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(prolog,A).
   paradigma(prolog,logico)
?- paradigma(A,funcional).
   paradigma(scheme,funcional)
?- paradigma(java,logico).
   false.
?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
```

**0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?**

paradigma(prolog, P) pergunta qual é o paradigma do Prolog (devolve P = logico). paradigma(prolog, logico) pergunta se isso é verdade (só true ou false).

**0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.**

Significa que o programa não sabe isso, ou seja, não consegue provar. Vale o mundo fechado: o que não está na base é tratado como falso.

## Etapa 1: Uma base de fatos e perguntas sobre ela

**Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?**

mulher(Quem) tem 7 respostas.

(mae(M,F), mulher(F)) tem 4 pares: lucia–tereza, marta–joana, tereza–bia, joana–ana.

**Saída do familia.pl**

```
1. Saída:
--- 1. sem variavel: a resposta e true ou false
?- mae(joana,jake).
   mae(joana,jake)
?- mae(jake,joana).
   false.
?- pai(maria,jake).
   false.
--- 2. com variavel: o Prolog procura os valores que tornam a consulta verdadeira
?- mae(joana,A).
   mae(joana,jake)
   mae(joana,ana)
?- pai(A,jake).
   pai(jorge,jake)
?- mulher(A).
   mulher(lucia)
   mulher(marta)
   mulher(tereza)
   mulher(joana)
   mulher(bia)
   mulher(ana)
   mulher(dora)
--- 3. conjuncao (virgula = E): as duas metas valem com o mesmo valor
?- mae(joana,A),homem(A).
   mae(joana,jake),homem(jake)
?- pai(vern,A),mulher(A).
   pai(vern,joana),mulher(joana)
?- mae(A,B),mulher(B).
   mae(lucia,tereza),mulher(tereza)
   mae(marta,joana),mulher(joana)
   mae(tereza,bia),mulher(bia)
   mae(joana,ana),mulher(ana)

2. Saída:
--- 1. sem variavel: a resposta e true ou false
?- mae(joana,jake).
   mae(joana,jake)
?- mae(jake,joana).
   false.
?- pai(maria,jake).
   false.
--- 2. com variavel: o Prolog procura os valores que tornam a consulta verdadeira
?- mae(joana,A).
   mae(joana,jake)
   mae(joana,ana)
?- pai(A,jake).
   pai(jorge,jake)
?- mulher(A).
   mulher(lucia)
   mulher(marta)
   mulher(tereza)
   mulher(joana)
```

```
   mulher(bia)
   mulher(ana)
   mulher(dora)
--- 3. conjuncao (virgula = E): as duas metas valem com o mesmo valor
?- mae(joana,A),homem(A).
   mae(joana,jake),homem(jake)
?- pai(vern,A),mulher(A).
   pai(vern,joana),mulher(joana)
?- mae(A,B),mulher(B).
   mae(lucia,tereza),mulher(tereza)
   mae(marta,joana),mulher(joana)
   mae(tereza,bia),mulher(bia)
   mae(joana,ana),mulher(ana)
?- homem(A),mulher(A).
   false.
```

**1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.**

“Mae é mãe de Filha e Filha é mulher”, ou seja, as mães e suas filhas. Teve 4 respostas.

**1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?**

É a hipótese do mundo fechado. O problema é que ela confunde “não sei” com “falso”. Em situações reais a base quase nunca é completa.

**1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?**

A ordem foi lucia, marta, tereza, joana, bia, ana, dora. É a ordem dos fatos no arquivo, porque o Prolog busca de cima para baixo.

## Etapa 2: Regras: ensinando o Prolog a deduzir

**Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?**

tio(X, jake) dá só X = paulo. primo(leo, X) dá jake e ana.

**Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?**

f(X,X)=f(a,b), [A|B]=[] e 2+3=5. Só f(X,X)=f(a,Y) unifica.

**Saídas de regras.pl e unificacao.pl**

```
1. Saída:
--- avo: duas regras progenitor encadeadas
?- avo(A,jake).
   avo(marta,jake)
   avo(vern,jake)
?- avo(osvaldo,A).
   avo(osvaldo,joana)
   avo(osvaldo,paulo)
   avo(osvaldo,bia)
--- irmao, tio e primo
?- irmao(jake,A).
   irmao(jake,ana)
?- tio(A,jake).
   tio(paulo,jake)
?- primo(leo,A).
   primo(leo,jake)
```

```
   primo(leo,ana)
--- ancestral (recursiva)
?- ancestral(osvaldo,A).
   ancestral(osvaldo,vern)
   ancestral(osvaldo,tereza)
   ancestral(osvaldo,joana)
   ancestral(osvaldo,paulo)
   ancestral(osvaldo,jake)
   ancestral(osvaldo,ana)
   ancestral(osvaldo,leo)
   ancestral(osvaldo,bia)
?- ancestral(A,jake).
   ancestral(joana,jake)
   ancestral(jorge,jake)
   ancestral(lucia,jake)
   ancestral(marta,jake)
   ancestral(osvaldo,jake)
   ancestral(vern,jake)
?- ancestral(jake,A).
   false.

2. Saída:
--- constantes e variaveis
a  =  a    unifica
a  =  b    NAO unifica
A  =  a    unifica   A = a
--- estruturas
f(A,b)  =  f(a,B)    unifica   A = a   B = b
f(A,A)  =  f(a,b)    NAO unifica
f(A,A)  =  f(a,B)    unifica   A = a   B = a
pai(A,jake)  =  pai(jorge,B)    unifica   A = jorge   B = jake
--- listas: [Cabeca|Cauda]
[A|B]  =  [1,2,3]    unifica   A = 1   B = [2,3]
[A|B]  =  []    NAO unifica
[A,B|C]  =  [a,b,c,d]    unifica   A = a   B = b   C = [c,d]
--- aritmetica NAO e unificacao
2+3  =  5    NAO unifica
g(A,b)  =  g(a,A)    NAO unifica
```

**2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?**

O caso base (progenitor) pega pai ou mãe direto. O recursivo sobe uma geração e chama ancestral de novo. Sem o base, a recursão nunca termina; sem o recursivo, só acharia pais e mães.

**2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?**

Sem X \== Y, irmao(jake, X) também devolve X = jake, porque todo mundo tem a mesma mãe e o mesmo pai que si mesmo.

**2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?**

No primeiro caso X teria de ser a e b ao mesmo tempo. No segundo, X = a e Y = a resolvem.

**2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.**

Não, uma função de Java ou Scheme tem entradas e saída fixas. No Prolog avo é uma relação, e qualquer argumento pode ser a variável.

## Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

**Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?**

concatena(X, Y, [1,2,3]) dá 4 respostas: [] e [1,2,3], [1] e [2,3], [1,2] e [3], [1,2,3] e [].

**Saída do listas.pl**

```
1. Saída:
--- membro: serve para testar E para gerar
?- membro(b,[a,b,c]).
   membro(b,[a,b,c])
?- membro(z,[a,b,c]).
   false.
?- membro(A,[a,b,c]).
   membro(a,[a,b,c])
   membro(b,[a,b,c])
   membro(c,[a,b,c])
--- tamanho
?- tamanho([a,b,c],A).
   tamanho([a,b,c],3)
?- tamanho([],A).
   tamanho([],0)
--- concatena: o mesmo codigo, em varias direcoes
?- concatena([1,2],[3,4],A).
   concatena([1,2],[3,4],[1,2,3,4])
?- concatena(A,[3],[1,2,3]).
   concatena([1,2],[3],[1,2,3])
?- concatena([1],A,[1,2,3]).
   concatena([1],[2,3],[1,2,3])
?- concatena(A,B,[1,2,3]).
   concatena([],[1,2,3],[1,2,3])
   concatena([1],[2,3],[1,2,3])
   concatena([1,2],[3],[1,2,3])
   concatena([1,2,3],[],[1,2,3])
--- inverte
?- inverte([1,2,3],A).
   inverte([1,2,3],[3,2,1])

2. Saída:
--- membro: serve para testar E para gerar
?- membro(b,[a,b,c]).
   membro(b,[a,b,c])
?- membro(z,[a,b,c]).
   false.
?- membro(A,[a,b,c]).
   membro(a,[a,b,c])
   membro(b,[a,b,c])
   membro(c,[a,b,c])
--- tamanho
?- tamanho([a,b,c],A).
   tamanho([a,b,c],3)
?- tamanho([],A).
   tamanho([],0)
--- concatena: o mesmo codigo, em varias direcoes
?- concatena([1,2],[3,4],A).
   concatena([1,2],[3,4],[1,2,3,4])
?- concatena(A,[3],[1,2,3]).
```

```
   concatena([1,2],[3],[1,2,3])
?- concatena([1],A,[1,2,3]).
   concatena([1],[2,3],[1,2,3])
?- concatena(A,B,[1,2,3]).
   concatena([],[1,2,3],[1,2,3])
   concatena([1],[2,3],[1,2,3])
   concatena([1,2],[3],[1,2,3])
   concatena([1,2,3],[],[1,2,3])
--- inverte
?- inverte([1,2,3],A).
   inverte([1,2,3],[3,2,1])
?- membro(A,[]).
   false.
```

**3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?**

O caso base é tamanho([], 0). O passo recursivo é tamanho([_|C], N) :- tamanho(C, M), N is M+1. A cada chamada fica pendente o N is M+1, que só é calculado na volta da recursão.

**3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?**

São 4 porque uma lista de 3 elementos tem 4 pontos de corte (n+1). O anexar do Scheme só junta duas listas já dadas e não consegue separar.

**3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?**

Foi a unificação mais o backtracking. A 1ª cláusula unifica X com a cabeça. Ao voltar, a 2ª recursa na cauda, e assim por diante.

## Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

**Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?**

X = 2+3 mostra X = 2+3 (estrutura). X is 2+3 dá X = 5. fatorial(N, 120) dá erro de instanciação.

**Saída do aritmetica.pl**

```
1. Saída:
--- = unifica, is calcula
?- A=2+3.
   2+3=2+3
?- A is 2+3.
   5 is 2+3
?- 5=:=2+3.
   5=:=2+3
?- 2+3=5.
   false.
?- A is 7 mod 3.
   1 is 7 mod 3
?- A is 7//2.
   3 is 7//2
?- A is 2^10.
   1024 is 2^10
--- fatorial, fib e soma
?- fatorial(5,A).
   fatorial(5,120)
?- fatorial(0,A).
   fatorial(0,1)
```

```
?- fib(10,A).
   fib(10,55)
?- soma([1,2,3,4],A).
   soma([1,2,3,4],10)
--- aritmetica nao e relacional
?- fatorial(A,120).
   ERRO: instantiation_error
?- A is B+1.
   ERRO: instantiation_error

2. Saída:
--- = unifica, is calcula
?- A=2+3.
   2+3=2+3
?- A is 2+3.
   5 is 2+3
?- 5=:=2+3.
   5=:=2+3
?- 2+3=5.
   false.
?- A is 7 mod 3.
   1 is 7 mod 3
?- A is 7//2.
   3 is 7//2
?- A is 2^10.
   1024 is 2^10
--- fatorial, fib e soma
?- fatorial(5,A).
   fatorial(5,120)
?- fatorial(0,A).
   fatorial(0,1)
?- fib(10,A).
   fib(10,55)
?- soma([1,2,3,4],A).
   soma([1,2,3,4],10)
--- aritmetica nao e relacional
?- fatorial(A,120).
   ERRO: instantiation_error
?- A is B+1.
   ERRO: instantiation_error
?- fatorial(10,A).
   fatorial(10,3628800)
```

**4.1 Explique com exemplos a diferença entre =, is e =:=.**

= unifica sem calcular (2+3 = 5 é falso). is avalia o lado direito e unifica com o esquerdo (X is 2+3 dá X = 5). =:= compara valores numéricos dos dois lados (5 =:= 2+3 é verdadeiro).

**4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?**

N > 0 e is exigem valores já conhecidos, e o Prolog não resolve equações ao contrário. concatena usa só unificação, que funciona em qualquer direção.

**4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?**

A estrutura é igual: caso base 0→1 e caso recursivo n * fact(n-1). Em Prolog o resultado é um argumento extra, é preciso o teste N > 0, o is e variáveis intermediárias, e só funciona em um sentido.

## Etapa 5: Controlando (e sofrendo com) a busca

**Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?**

solteiro2 dá false. maximo2(5,3,M) dá duas respostas (5 e 3). maximo3(5,3,3) dá true, que é errado.

**Saída de controle.pl e mensagem de ancestral_ruim**

```
Saída controle.pl:
--- A) backtracking: acompanhe a ordem das tentativas
?- mesmo_gosto(X, Y).
   tenta ana e ana
   tenta ana e bia
   tenta ana e caio
   resposta: mesmo_gosto(ana,caio)
   tenta bia e ana
   tenta bia e bia
   tenta bia e caio
   tenta caio e ana
   resposta: mesmo_gosto(caio,ana)
   tenta caio e bia
   tenta caio e caio
--- B) negacao: a ordem das metas muda a resposta
?- solteiro1(A).
   solteiro1(osvaldo)
   solteiro1(paulo)
   solteiro1(jake)
?- solteiro2(A).
   false.
--- C) corte
?- maximo1(5,3,A).
   maximo1(5,3,5)
?- maximo2(5,3,A).
   maximo2(5,3,5)
   maximo2(5,3,3)
?- maximo3(5,3,A).
   maximo3(5,3,5)
?- maximo1(5,3,3).
   false.
?- maximo3(5,3,3).
   maximo3(5,3,3)

Saída laco_infinito.pl:
ancestral_bom(osvaldo, jake)?
  sim
ancestral_ruim(osvaldo, jake)? (vai estourar a pilha)

Fatal Error: local stack overflow (size: 16384 Kb, reached: 16383 Kb, environment variable
used: LOCALSZ)
```

**Previsão: Quantas colorações existem com 3 cores? E com 2 cores?**

Com 3 cores existem 24 colorações. Com 2 cores, nenhuma (false), porque PR, SP e MS fazem fronteira entre si e formam um triângulo, então precisam de 3 cores diferentes.

**Saída do mapa.pl**

```
--- coloracoes com 3 cores, na ordem [RS, SC, PR, SP, MS, MG, RJ, ES]
[vermelho,verde,vermelho,verde,azul,vermelho,azul,verde]
[vermelho,verde,vermelho,azul,verde,vermelho,verde,azul]
--- quantas coloracoes existem com 3 cores?
24
--- e com 2 cores?
?- mapa([vermelho,verde],A).
   false.
--- e com 4 cores?
1728
```

**5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?**

Em tenta ana e ana falhou X \== Y, e o Prolog voltou ao último ponto de escolha, amigo(Y), tentando bia. Em ana e bia falhou gosta(bia, cafe). Em ana e caio deu certo, resposta (ana, caio). Mais adiante vem (caio, ana).

**5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?**

Em solteiro2, X está livre e \+ casado(X) falha de imediato, já que casado(vern) é provável. Em solteiro1 o homem(X) gera X antes. Osvaldo “é solteiro” porque a base não tem casado(osvaldo) (mundo fechado: não consegue provar, então a negação vale).

**5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?**

Com o 3º argumento já 3, a cabeça maximo3(X,Y,X) nem unifica (5 ≠ 3), então o corte nunca é executado. A 2ª cláusula aceita. Correção:
maximo3(X, Y, Z) :- X >= Y, !, Z = X.
maximo3(_, Y, Y).

**5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?**

ancestral_ruim chama a si mesma primeiro, com os mesmos argumentos, e nunca consome um fato. A busca é em profundidade e a ordem das metas importa. As duas são equivalentes na lógica, mas não no controle.

**5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?**

Com 3 cores são 24. Com 2 cores, 0 (false), pois PR, SP e MS formam um triângulo e precisam de 3 cores. Com 4 cores, 1728 (calculei à mão: 24·8·3·3). Confira na sua saída. É declarativo porque você só descreve o que é uma coloração válida, e quem busca é o Prolog.

## Fechamento: O que você viu e onde isso se encaixa

**6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.**

No funcional, anexar é uma função com entrada e saída fixas (só junta). No lógico, concatena é uma relação, então serve para juntar, separar e testar com o mesmo código.

**6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.**

(1) Dependência da ordem das metas e recursão à esquerda: ancestral_ruim estourou a pilha. (2) Negação como falha e mundo fechado: solteiro1 disse que osvaldo é solteiro. Também vale a aritmética não relacional (fatorial(N,120)) e o corte (maximo3).

**6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?**

Eu escolheria Prolog para busca, restrições, bancos dedutivos, sistemas especialistas e linguagem natural. Não escolheria para cálculo numérico pesado, interface gráfica ou programas com muito estado e E/S.